The Emergent State Machine: A Turn-Based Control Architecture

Richard Hines
Independent Researcher
Tacoma, Washington, USA
richardhinestacoma@gmail.com

Abstract

Contemporary AI systems often entangle probabilistic interpretation with consequential action selection. When the same mechanism both describes state and authorizes action, authority becomes implicit, drift becomes opaque, and accountability weakens.

This paper argues that AI systems should structurally separate descriptive state representation from authoritative action selection. We introduce the Emergent State Machine (ESM), a control architecture in which raw input is projected into an explicit, versioned state space, and action selection is performed exclusively by a deterministic, versioned policy. Generative components, when present, are confined to non-authoritative, schema-constrained roles.

This separation yields replayability, failure localization, and controlled evolution under what we term Instrumented Deterministic Evolution (IDE). While motivated by AI systems, the principle generalizes to any automated system in which probabilistic description and consequential action coexist.

A reference implementation is available at: github.com/ih8scargo/esm.

1 Introduction

Modern AI systems increasingly combine probabilistic interpretation with consequential action selection. Large language models and related generative systems are frequently used not only to describe or summarize information, but to route workflows, approve transactions, escalate risk, or determine user-facing outcomes. In many such systems, interpretation and authority are implemented within the same probabilistic mechanism.

When the same model both describes state and selects action, the boundary between description and authority collapses. Policy becomes implicit in model weights rather than explicit in versioned artifacts. Behavioral drift may occur through parameter updates without corresponding changes to declared decision rules. Failure localization becomes diffuse, spanning representation, inference, and action simultaneously.

We argue that AI systems should structurally separate descriptive state representation from authoritative action selection. Interpretation may remain probabilistic; authority over consequential action should not. Systems that entangle description and authority risk obscuring accountability boundaries and weakening governance guarantees.

The principle advanced here is consistent with longstanding architectural disciplines. Software engineering formalizes separation of concerns. The model–view–controller pattern distinguishes representation from control. Classical control theory separates state estimation from control input. Constitutional governance separates deliberation from execution. In each case, system robustness depends on preventing descriptive mechanisms from directly exercising authority.

The Emergent State Machine (ESM) extends this principle into the design of modern AI and automated decision systems. It enforces a non-bypassable boundary between:

- Description: explicit projection of raw input into structured state, and
- Authority: deterministic policy governing action selection.

Generative components, when present, may assist in interpretation but cannot authorize actions.

The distinction between entangled generative systems and an ESM can be summarized structurally:

Entangled Generative System	**Emergent State Machine (ESM)**
Interpretation and action selection occur within the same probabilistic mechanism.	Descriptive projection and authoritative policy are structurally separated.
Authority is embedded in model parameters.	Authority resides exclusively in deterministic, versioned policy artifacts.
Ambiguity is collapsed internally through probabilistic inference.	Ambiguity triggers explicit clarification and state acquisition.
Behavioral drift may occur through model updates without explicit rule change.	Behavior changes only through explicit revision of projection or policy versions.
Failure localization spans representation and decision logic simultaneously.	Failures are localizable to projection, policy, or clarification boundary.

Table 1: Structural comparison between entangled generative architectures and the Emergent State Machine.

In entangled systems, interpretation and authority are inseparable. The same probabilistic mechanism both represents state and determines action. Behavioral characteristics are embedded within model parameters, and drift occurs implicitly through model updates.

An ESM enforces structural separation. State is explicit and versioned. Policy is deterministic and versioned. Ambiguity is not internally collapsed but triggers clarification and state acquisition. Behavioral change occurs only through explicit revision of projection or policy artifacts.

The remainder of this paper formalizes this separation principle. Section 2 defines the architectural components of the ESM. Section 3 analyzes the stability properties that follow from authority separation. Section 4 describes instrumentation and versioning under Instrumented Deterministic Evolution (IDE). Section 5 examines architectural affordances. Section 6 discusses systems engineering implications. Section 7 outlines limitations. Section 8 provides a reference implementation. Section 9 concludes.

2 Formal Architecture

An Emergent State Machine (ESM) enforces a structural boundary between descriptive interpretation and authoritative action selection. Projection may describe state; generative components

may assist interpretation; only deterministic policy may authorize action.

An ESM consists of three primary components:

- A Projection Operator R
- A Deterministic Policy π
- An optional Schema-Constrained Generative Instrument G

At its core, execution follows:

$$I_t \xrightarrow{R} x_t \xrightarrow{\pi} a_t$$

where I_t is raw input, x_t is explicit state, and a_t is an authorized action.

2.1 Structural Separation

The upper layer performs description. Raw input is projected into explicit, versioned state. Generative components may assist in structured interpretation but remain non-authoritative.

The lower layer performs authority. Only deterministic policy may select actions. The authority boundary is non-bypassable: no generative or interpretive mechanism may directly produce actions.

This separation is the central invariant of the ESM.

2.2 Projection Operator (R)

The projection operator transforms raw input into bounded, explicit state:

$$x_t = R_v(I_t)$$

where:

- I_t may include user input, environmental signals, and prior interaction context.
- $x_t \in \mathcal{X}$ is a structured state vector composed of explicitly defined primitives.
- v denotes the projection version.

Projection has four defining properties:

1. **Explicit Primitives**
   Each component of x_t corresponds to a documented primitive.
2. **Bounded State Space**
   The dimensionality of $\mathcal{X}$ is finite and interpretable.

3. **Determinism**
   For fixed v, identical input yields identical state.

4. **Versioning**
   Any change to primitive definitions, thresholds, or aggregation rules produces a new projection version.

Projection describes state. It does not authorize action.

2.3 Deterministic Policy (π)

The deterministic policy maps state to action:

$$a_t = \pi_w(x_t)$$

where:

- $a_t \in \mathcal{A}$ is a member of a finite action set.
- w denotes the policy version.

Policy has four defining properties:

1. **Explicit Mapping**
   Decision logic is encoded as threshold rules, branching logic, or deterministic lookup.

2. **Determinism**
   For fixed w, identical state yields identical action.

3. **Authority**
   Policy is the sole mechanism authorized to select actions.

4. **Versioning**
   Any change to decision rules produces a new policy version.

Authority resides exclusively in π.

2.4 Clarification Under Partial Observability

An ESM assumes that projected state may be incomplete. When x_t does not yield a unique admissible action under policy, the system does not collapse ambiguity internally through probabilistic inference. Instead, ambiguity triggers explicit state acquisition.

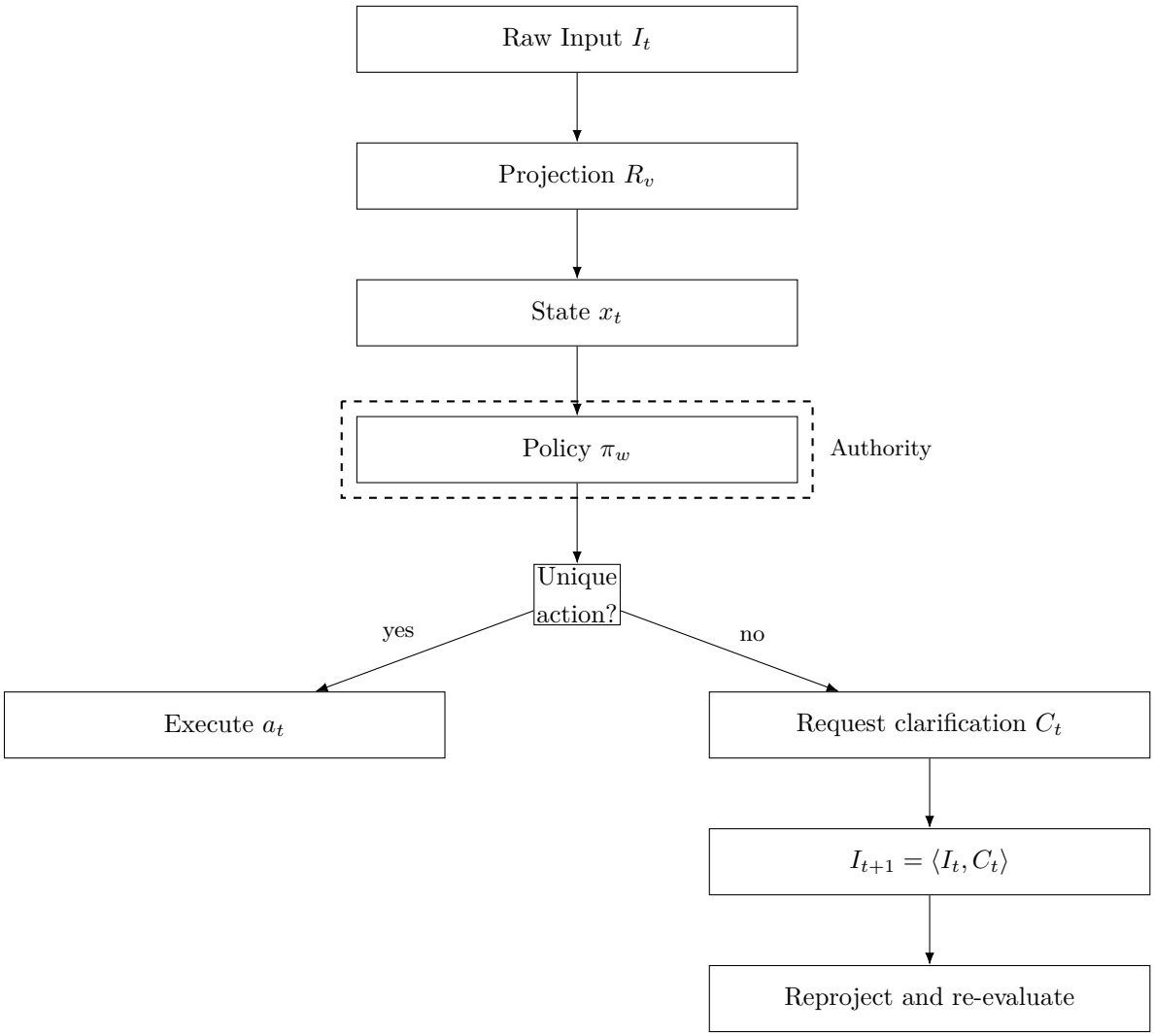

Figure 1: Clarification under partial observability. If projected state does not yield a unique admissible action, policy requests clarification and the system reprojects and re-evaluates deterministically.

If no unique action is admissible:

$$\pi_w(x_t) = \text{request_clarification}$$

Clarification input C_t augments system input:

$$I_{t+1} = \langle I_t, C_t \rangle$$

The system then reprojects and re-evaluates:

$$x_{t+1} = R_v(I_{t+1}), \qquad a_{t+1} = \pi_w(x_{t+1})$$

Ambiguity is not collapsed internally. It triggers explicit state acquisition. Interpretation may be probabilistic; authority remains deterministic.

2.5 Schema-Constrained Generative Instrument (Optional)

A generative component G may assist projection or clarification formatting:

$$y_t = G(x_t, s)$$

where s is a predefined schema constraining output structure.
Generative outputs must:

- Validate against schema.
- Remain non-authoritative.
- Pass through policy before action selection.

The generative instrument cannot bypass the authority boundary.

2.6 Execution Paths

An ESM supports two primary execution paths.

Deterministic Baseline Path

$$I_t \xrightarrow{R_v} x_t \xrightarrow{\pi_w} a_t$$

Clarification Path

$$I_t \xrightarrow{R_v} x_t \xrightarrow{\pi_w} \text{clarify}$$

$$I_{t+1} \xrightarrow{R_v} x_{t+1} \xrightarrow{\pi_w} a_{t+1}$$

In both cases, policy remains the sole authority for action selection.

3 Stability Consequences of Authority Separation

The structural separation between description and authority produces stability properties that are difficult to obtain in entangled probabilistic systems. These properties do not arise

from determinism alone, but from the explicit isolation of authoritative action selection from interpretive mechanisms.

3.1 Deterministic Action Selection

For fixed projection version v and policy version w,

$$I_t \xrightarrow{R_v} x_t \xrightarrow{\pi_w} a_t$$

is deterministic.

Identical inputs under identical versions yield identical actions.

Because generative components are non-authoritative, variability in probabilistic interpretation cannot directly alter action selection. Any behavioral change must arise from explicit revision of projection or policy artifacts.

Determinism here refers to version stability, not epistemic certainty. The system may operate under uncertainty, but action selection is stable under fixed structural definitions.

3.2 Replayability and Regression Testing

Authority separation enables exact replay.

Given stored:

- Raw input I_t
- Projection version R_v
- Policy version π_w

the resulting action a_t can be recomputed exactly.

Formally, under fixed versions:

$$a_t = \pi_w(R_v(I_t))$$

This permits:

- Regression testing prior to deployment of new versions
- Cross-version behavioral comparison
- Counterfactual analysis under alternative policy definitions

Because authority is encoded explicitly, behavioral drift cannot occur without identifiable artifact change.

3.3 Failure Localization

When interpretation and authority are entangled, undesirable outcomes may originate from diffuse interactions between representation and decision logic.

In an ESM, failure must arise from one of three loci:

- Projection definition
- Policy definition
- Clarification boundary specification

Since generative components cannot authorize actions, they cannot independently produce unauthorized behavior.

This sharply reduces diagnostic ambiguity.

3.4 Partial Observability and Clarification

The ESM assumes that projected state may be incomplete. Under partial observability, unique action selection may not be possible from available state.

Rather than collapsing ambiguity internally, policy selects clarification as an action:

$$\pi_w(x_t) = \text{request_clarification}$$

Clarification increases observability through additional input, after which projection and policy re-evaluate deterministically.

Ambiguity is not collapsed internally. It triggers explicit state acquisition.

This preserves authority separation while acknowledging epistemic limits.

3.5 Stability Under Temporal Accumulation

Long-horizon interactions may incorporate historical aggregates within projected state. However, any temporal accumulation, decay, or reset logic must be explicitly encoded within projection.

There is no implicit mutation of policy over time.

System evolution may occur across versions, but mechanism stability is preserved within versions.

4 Instrumentation and Versioning

The separation of description and authority is enforceable only if both layers are explicit and versioned. The Emergent State Machine therefore operates under a regime of structured instrumentation and controlled revision.

Behavioral change must be attributable to identifiable artifact modification. Implicit evolution through parameter drift is incompatible with authority separation.

4.1 Logged Interaction Trace

Each interaction produces a structured trace containing:

- Raw input I_t
- Projection version v
- Projected state x_t
- Policy version w
- Selected action a_t
- Clarification events, if triggered

Because clarification is a first-class action, ambiguity detection becomes visible and auditable. The system record distinguishes between:

- Direct action selection
- Ambiguity-triggered clarification
- Post-clarification action selection

Ambiguity is therefore an observable event rather than an internal inference.

4.2 Projection Versioning

Projection is a versioned artifact.

Any modification to:

- Primitive definitions
- Threshold values
- Feature aggregation logic
- Temporal accumulation rules

produces a new projection version R_{v+1}.

State interpretation changes are therefore explicit and attributable. Historical interactions can be replayed under prior projection versions to evaluate behavioral impact.

4.3 Policy Versioning

Policy is likewise versioned.

Any change to:

- Decision thresholds
- Routing logic
- Escalation rules
- Clarification triggers

produces a new policy version π_{w+1}.

Since policy exclusively authorizes action, any change in system behavior must correspond to explicit policy revision or projection revision.

Authority cannot drift silently.

4.4 Cross-Version Replay

Given stored interaction traces and version identifiers, system behavior can be reproduced exactly:

$$(I_t, R_v, \pi_w) \;\rightarrow\; a_t$$

This supports:

- Regression testing before deployment
- Behavioral comparison across versions
- Controlled experimentation
- Audit and compliance review

Replayability follows directly from deterministic authority separation.

4.5 Instrumented Deterministic Evolution (IDE)

We refer to this regime as *Instrumented Deterministic Evolution (IDE)*.

Under IDE:

- System behavior changes only through explicit revision of versioned projection or policy artifacts.

- No implicit authority evolution occurs through probabilistic parameter updates.
- Mechanism stability is preserved within versions.

IDE does not require convergence of outcomes. Outcomes may change across versions. What remains stable is the structural pathway by which actions are authorized.

Separation of description and authority makes evolution intentional rather than emergent.

5 Design Affordances

The separation of description and authority enables architectural capabilities that are difficult to achieve in entangled probabilistic systems. These affordances arise from explicit state representation, deterministic policy, and versioned evolution.

5.1 Multi-Policy Coexistence

Because projection produces a shared, explicit state space, multiple policies may operate over the same state representation:

$$a_t = \pi_i(x_t)$$

Distinct policies may encode:

- Different risk tolerances
- Alternative governance constraints
- Organizational preferences
- Experimental decision rules

Policy selection itself may be governed by a higher-level deterministic rule.

Since authority is isolated within policy artifacts, comparative evaluation and controlled experimentation can occur without modifying projection or generative components.

5.2 Controlled Ambiguity Handling

Ambiguity is formally defined as the absence of a unique admissible action under current policy constraints.

Rather than collapsing ambiguity internally, the ESM selects clarification as an explicit action. This provides:

- Observability of ambiguity conditions

- Explicit logging of underdetermined states
- Deterministic handling of partial observability

Ambiguity therefore becomes a measurable system property rather than an implicit inference artifact.

5.3 Temporal Depth Management

Long-horizon systems often incorporate historical context. In an ESM, historical aggregates may be encoded explicitly within projected state.

However, any temporal accumulation, decay, or reset rule must be defined within projection and versioned accordingly.

There is no implicit memory drift.

Temporal influence remains inspectable and revisable.

5.4 Meta-Policy Layer

Because authority is isolated within policy artifacts, higher-order governance logic can operate over policies themselves.

A deterministic meta-policy may govern:

- Policy version selection
- Deployment conditions
- Rollback triggers
- Controlled A/B evaluation

Meta-policy does not modify projection. It selects among explicit policy artifacts.

This preserves authority separation across organizational layers.

5.5 Instrumented Schema Discovery

Generative mechanisms may assist in proposing candidate projection refinements offline.

For example:

- Identifying recurring structural patterns
- Suggesting new classification primitives
- Proposing alternative decompositions of state

Such proposals must:

- Conform to explicit schema definitions.
- Be validated offline.
- Undergo regression testing.
- Be integrated through a new projection version if adopted.

Generative assistance may inform description. It may not alter authority without explicit revision.

6 Systems Engineering Implications

The separation of description and authority has implications beyond conversational AI. Any automated system that combines probabilistic interpretation with consequential action may benefit from explicit authority boundaries.

6.1 Distributed Deployment

In distributed systems, components responsible for interpretation and action often operate across different services or environments.

An ESM enables:

- Projection at the edge or client layer
- Centralized policy enforcement
- Sandboxed generative assistance

Because authority resides solely in policy, distributed execution does not fragment control. State representation remains explicit, and action authorization remains deterministic.

This separation supports federated architectures in which local agents compute structured state while centralized policy governs consequential action.

6.2 Governance and Audit

Systems operating in regulated or high-accountability contexts require traceability.

An ESM provides:

- Explicit projected state
- Version identifiers for projection and policy
- Logged ambiguity detection

- Logged clarification actions
- Deterministic action traces

Audit does not require inspection of model weights. It requires replay under stored versions.

Authority separation transforms decision systems into inspectable control systems.

6.3 Controlled Evolution Pipelines

Because projection and policy are explicit artifacts, deployment pipelines can incorporate structured validation.

For example:

1. Propose projection revision R_{v+1}.
2. Replay historical interaction corpus.
3. Measure behavioral deltas.
4. Approve or reject revision.

The same applies to policy updates.

Behavioral change becomes measurable and intentional rather than emergent from parameter updates.

6.4 Example: Intent Clarification in Service Systems

Consider a service-routing system receiving underdetermined input such as "I need help with my account."

In entangled architectures, probabilistic inference collapses intent internally and routes immediately. When intent confidence is marginal, misrouting may occur without explicit indication of ambiguity.

In an ESM:

- Projection produces structured intent scores.
- Policy evaluates determinacy conditions.
- If no unique admissible routing action exists, policy selects:

$$a_t = \text{request_clarification}$$

Clarification input augments system state.

Projection and policy re-evaluate deterministically.

Ambiguity is surfaced rather than absorbed. Routing decisions become traceable to explicit state and policy logic.

The same principle applies to:

- Educational tutoring systems
- Risk assessment workflows
- Compliance routing engines
- Automated governance layers

In each case, separation of description and authority reduces implicit decision drift and improves diagnosability.

7 Limitations and Boundaries

The Emergent State Machine enforces structural separation between description and authority. It does not eliminate the inherent limitations of modeling, interpretation, or governance.

7.1 Projection Quality

Projection determines how raw input is represented as state. If primitives are poorly defined, incomplete, or biased, deterministic policy will faithfully amplify those deficiencies.

The ESM does not guarantee that state representation is correct. It guarantees only that representation is explicit and versioned.

7.2 Policy Design

Deterministic policy may encode flawed thresholds, inappropriate escalation logic, or undesirable governance assumptions.

Authority separation makes such issues inspectable and revisable. It does not prevent them.

Transparency does not imply correctness.

7.3 Partial Observability Persists

Clarification increases observability but does not eliminate epistemic limits. Some domains may remain underdetermined even after additional input.

The ESM treats ambiguity as a boundary condition and surfaces it explicitly. It does not guarantee complete state knowledge.

7.4 Performance Trade-Offs

Explicit clarification and deterministic policy evaluation may introduce additional interaction steps relative to architectures that collapse ambiguity internally.

The ESM prioritizes control integrity and traceability over conversational smoothness. In domains where minimal latency or maximal generative fluidity is the primary objective, full authority separation may not be desirable.

7.5 Applicability Scope

The ESM is most applicable in systems where consequential action extends beyond text generation itself. In purely generative contexts—such as creative writing tools—interpretation and output may coincide, and authority separation may be unnecessary.

The architecture is therefore a discipline for systems in which probabilistic interpretation influences externally consequential decisions.

8 Reference Implementation

This section outlines a minimal reference structure sufficient to instantiate the architectural invariants described above. The implementation is illustrative rather than exhaustive.

8.1 Core Components

A minimal ESM implementation consists of four modules:

Projection Engine Deterministic transformation from raw input I_t to structured state x_t.

- Encodes primitive definitions
- Applies threshold and aggregation rules
- Maintains projection version identifier

Policy Engine Deterministic mapping from state x_t to action a_t.

- Encodes routing and escalation logic
- Detects ambiguity conditions
- Selects clarification actions when necessary
- Maintains policy version identifier

Clarification Interface Mechanism for acquiring additional input when policy selects a clarification action.

- May incorporate generative assistance for formatting
- Returns structured clarification input C_t

Instrumentation Layer Logs interaction traces including:

- Raw input
- Projected state
- Policy decision
- Clarification events
- Version identifiers

These components enforce the non-bypassable authority boundary.

8.2 Deterministic Execution Flow

Baseline Execution

$$I_t \xrightarrow{R_v} x_t \xrightarrow{\pi_w} a_t$$

Clarification Execution

$$I_t \xrightarrow{R_v} x_t \xrightarrow{\pi_w} \text{clarify}$$

$$I_{t+1} = \langle I_t, C_t \rangle \xrightarrow{R_v} x_{t+1} \xrightarrow{\pi_w} a_{t+1}$$

At no point may a generative mechanism directly authorize action.

8.3 Illustrative Execution Fragment

```
x_t = R_v(I_t)
a_t = pi_w(x_t)

if a_t == "request_clarification":
    C_t = acquire_clarification()
    I_t1 = augment(I_t, C_t)
    x_t1 = R_v(I_t1)
```

```
    a_t1 = pi_w(x_t1)
    execute(a_t1)
else:
    execute(a_t)
```

This fragment highlights three invariants:

- Projection precedes authority.
- Policy exclusively selects actions.
- Clarification augments state rather than bypassing policy.

8.4 Domain Portability

The ESM is domain-agnostic at the architectural level. Portability requires only that:

- A finite, explicit state representation can be defined.
- An explicit action set can be enumerated.
- Ambiguity conditions can be formally specified.

Projection and policy definitions may vary by domain, but the authority boundary remains invariant.

9 Related Work

The separation principle advanced in this paper extends and formalizes architectural ideas that have appeared across multiple domains.

9.1 Separation of Concerns in Software Architecture

The idea that system robustness depends on modular separation is longstanding in software engineering. Dijkstra articulated the principle of separation of concerns as a foundational strategy for managing system complexity [Dijkstra, 1974]. Information hiding and modular decomposition further reinforced this architectural discipline [Parnas, 1972].

The Emergent State Machine applies this principle to AI decision systems by isolating descriptive state representation from authoritative action selection. While separation of concerns is well established, the ESM formalizes an enforceable boundary in contexts where probabilistic interpretation and consequential action are frequently entangled.

9.2 Estimation and Control in Classical Systems

Classical control theory distinguishes state estimation from control input [Kalman, 1960, Åström and Murray, 2010]. In control theory, a distinction is made between state estimation (observers) and control input (controllers). Observers infer system state from measurements, while controllers determine actions based on estimated state. Stability analysis often depends on this structural separation.

The ESM echoes this architecture: projection corresponds to explicit state construction, and policy corresponds to deterministic control. However, unlike classical observer–controller pairs, modern generative systems often collapse estimation and action into a single probabilistic mechanism. The ESM reinstates an explicit boundary in such systems.

9.3 Model–View–Controller and Interface Boundaries

Architectural patterns such as Model–View–Controller (MVC) distinguish between data representation, user interface, and control logic, thereby reducing coupling and improving maintainability [Reenskaug, 1979].

The ESM similarly separates descriptive representation from authoritative control. Unlike many contemporary AI systems, however, this boundary is non-bypassable: generative or interpretive components may assist representation but cannot directly authorize action.

9.4 Interpretability and AI Governance

Recent work in interpretability and governance emphasizes accountability in automated systems [Doshi-Velez and Kim, 2017, Mittelstadt et al., 2016]. Calls for explainable AI and traceable decision-making reflect growing concern over opaque model behavior.

The ESM contributes to this discourse by relocating authority from model weights to explicit, versioned policy artifacts. Rather than attempting to extract explanations from entangled models, it proposes architectural separation as a means of preserving accountability boundaries from the outset.

10 Conclusion

This paper advances a structural claim: systems that combine probabilistic interpretation with consequential action should separate description from authority.

When interpretation and action selection are entangled within a single probabilistic mechanism, authority becomes implicit. Behavioral drift may occur without corresponding artifact change, failure localization becomes diffuse, and governance boundaries weaken. These effects are not incidental implementation details; they arise from structural coupling.

The Emergent State Machine (ESM) formalizes a separation discipline. Raw input is projected into explicit, versioned state. Deterministic policy alone authorizes action. Generative components, when present, remain confined to descriptive or assistive roles and may not bypass the authority boundary. Ambiguity is not internally collapsed but treated as partial observability, triggering explicit state acquisition through clarification.

This separation yields replayability, diagnosability, and controlled evolution under Instrumented Deterministic Evolution (IDE). System behavior changes only through explicit revision of projection or policy artifacts. Authority does not drift silently.

While motivated by contemporary AI systems, the principle generalizes beyond them. Any automated or socio-technical system in which probabilistic interpretation influences externally consequential decisions benefits from explicit authority boundaries.

Separation of description and authority is not a safety feature. It is an architectural requirement for systems in which interpretation and action coexist.

References

Edsger W. Dijkstra. On the role of scientific thought. *Selected Writings on Computing: A Personal Perspective*, 1974. Introduced the principle of separation of concerns.

Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*, 2017.

Rudolf E. Kalman. A new approach to linear filtering and prediction problems. *Transactions of the ASME–Journal of Basic Engineering*, 82:35–45, 1960.

Brent Mittelstadt, Patrick Allo, Mariarosaria Taddeo, Sandra Wachter, and Luciano Floridi. The ethics of algorithms: Mapping the debate. *Big Data & Society*, 3(2), 2016.

David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

Trygve Reenskaug. Models - views - controllers. 1979. Technical note introducing MVC pattern.

Karl J. Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2010.